

Fractal Behavioral Grammar

A Compression-First Architecture for Local AI Personalization

Technical Whitepaper — Prior Art Disclosure — June 2026

Ryan Moore

github.com/ryandmoore1976/fractal-grammar · MIT License

v0.1.0 · fractal-grammar

Abstract

Current approaches to LLM personalization accumulate behavioral history and replay it as context. This paper argues that this is the wrong abstraction. Human behavioral sequences are not random — they exhibit measurable fractal self-similarity across timescales, a property quantified by the Hurst exponent (H). When $H > 0.70$, a compact set of generative rules — a behavioral grammar — can represent the full structure of an interaction log at a fraction of the storage cost. We describe a five-stage pipeline that extracts this grammar using MinHash LSH novelty gating, four-level HDBSCAN hierarchical clustering, and rescaled-range (R/S) Hurst analysis. On a 53-event accounting domain corpus we demonstrate 30× compression at 70ms CPU latency. We report stress-test results, including three falsifying findings from adversarial noise injection tests. We identify the minimum viable corpus size (150+ events with a semantic embedding backend) for trustworthy Hurst estimation. The pipeline is released as an open-source Python library under MIT license.

Keywords: fractal compression, behavioral grammar, Hurst exponent, local AI, LLM personalization, HDBSCAN, on-device AI, LoRA, privacy-first AI

1. The Problem with Storing Behavior

Every significant approach to LLM personalization today solves the same problem the same way: accumulate behavioral history and use it as context. The differences between systems are engineering choices — retrieval mechanism, context window size, compression format — but the underlying abstraction is identical: store what happened, retrieve what is relevant, inject it into the prompt.

This abstraction has a structural problem. It conflates two things the brain has always treated as categorically distinct: episodes and rules.

Neuroscience has established since the 1990s that the hippocampus (specifically CA1) encodes behavioral episodes — what happened, in sequence, with temporal specificity. The medial prefrontal cortex (mPFC) encodes behavioral rules — the abstract pattern that a class of episodes instantiates. These are not two views of the same data. They are two distinct representational systems with different compression properties. The rules system is compact by design. Rules generalize across episodes. The episode system is not compact — each episode is stored individually regardless of whether it contains new information.

Further, research on CA1 hippocampal neurons has shown that fractal spike timing patterns — not the count or density of spikes — predict faster learning. Specifically, longer-duration fractal patterns in CA1 correlate with accelerated behavioral acquisition. The brain does not learn faster by storing more; it learns faster when the temporal structure of stored patterns is fractal [PMC10217776].

Current LLM personalization builds only the episode system. It stores what happened. It does not ask whether the structure of what happened is compressible. It does not extract the rule. It stores the

instance.

The core claim of this paper: if behavioral interaction logs exhibit fractal self-similarity — which is measurable — then a compact behavioral grammar can represent those logs at a fraction of the storage cost without losing the structural information that drives personalization. Store the grammar. Not the log.

2. The Fractal Behavioral Grammar Hypothesis

A fractal process is one whose statistical structure is self-similar across timescales: the pattern you observe at coarse resolution predicts the pattern you observe at fine resolution. This property is quantified by the Hurst exponent H :

Hurst Exponent	Interpretation for Behavioral Logs
$H > 0.70$	Strong long-range correlation. Past predicts future across scales. Fractal structure present. Grammar compression is efficient.
$H \approx 0.50$	Random walk. No persistent structure. Compression provides no benefit.
$H < 0.50$	Anti-persistent (mean-reverting). Each observation opposes the prior. Grammar compression is inapplicable.

Table 1. Hurst exponent ranges and their implications for grammar compression viability.

Hypothesis: LLM interaction logs from domain-constrained AI assistants (accounting, legal, coding, medical) consistently exhibit $H > 0.50$ due to the inherent repetition of domain-specific behavioral patterns. When $H > 0.70$, a generative grammar — a compact set of rules that produces the observed behavioral structure — can represent the log with an order-of-magnitude compression ratio relative to storing individual events.

This hypothesis is falsifiable. The pipeline reports corpus H for any input log. If $H < 0.55$, the pipeline explicitly signals that grammar compression is not applicable and raw retrieval-augmented approaches are preferable. The Hurst exponent is the instrument; it is not assumed.

3. The Novel Combination

Each component of this pipeline draws from established fields. The novelty is the specific composition and its application to LLM behavioral personalization. Prior art on each component is acknowledged explicitly; the composition is, to the authors' knowledge, previously unpublished.

Component	Prior Art	Novel Application in This Work
Hurst exponent / R/S analysis	Mandelbrot & Wallis (1969). Applied to financial time series, network traffic, neural spike trains.	First application to LLM interaction log self-similarity as a compression criterion.
HDBSCAN hierarchical clustering	Campello et al. (2013). General-purpose density clustering algorithm.	First application across four behavioral resolution levels (domain → category → pattern → micro) with parent-child cross-level similarity tracing.
MinHash LSH novelty gating	Broder (1997). Near-duplicate detection in web crawls.	First use as a behavioral novelty gate controlling which LLM interactions enter the grammar pipeline.
LoRA delta adapters	Hu et al. (2021). Parameter-efficient fine-tuning.	Identified as the downstream consumer of grammar-derived training signal. Not implemented in v0.1.0 — identified as the integration target.
mPFC rule / CA1 episode distinction	Neuroscience literature, 1990s–present.	First use as the design rationale for separating behavioral grammar (rules) from raw interaction logs (episodes) in an AI personalization architecture.
Fractal spike timing → faster learning	PMC10217776, CA1 hippocampal research.	First citation as motivation for fractal-structured behavioral memory in ML systems.

Table 2. Component provenance and novel application. The prior art on each component is fully acknowledged. The novel claim is the pipeline composition and its application.

The composition that constitutes the novel claim: using Hurst exponent self-similarity detection across HDBSCAN resolution levels as the criterion for identifying compressible behavioral grammar rules from LLM interaction logs, motivated by the neuroscientific distinction between episodic and rule-based memory, for the purpose of compression-first local AI personalization. This combination has not appeared in the literature or in any productized tool as of the date of this publication.

4. Why Existing Approaches Fail at the Constraint Boundary

The constraint boundary for local AI personalization is memory. A consumer device running a local LLM has 8–16 GB of RAM of which the model occupies 4–8 GB. The personalization layer must operate in the remainder. Existing approaches hit this boundary in predictable ways.

Approach	Method	Compression	Failure Mode
Full context injection	Append raw interaction history to every prompt.	Zero.	Context window fills in 20–50 interactions. Latency increases linearly. No memory compression.

Approach	Method	Compression	Failure Mode
RAG (vector retrieval)	Embed interactions, retrieve top-k at inference time.	None (stores all embeddings).	Storage scales with interaction count. No structure-aware compression. Retrieval introduces latency.
LoRA fine-tuning (periodic)	Batch-train a LoRA adapter on accumulated logs.	Adapter weights (~5–50MB).	Raw logs must still be stored until training. No compression of the training signal itself. Catastrophic forgetting risk.
Prompt personalization	Maintain a user profile as structured text injected at system prompt.	None.	Manual curation. Does not capture behavioral patterns automatically. Not computable from logs.
Fractal Grammar (this work)	Extract generative rules from behavioral logs. Store rules, not logs.	30× on test corpus. Scales with behavioral novelty, not interaction count.	Requires $H > 0.55$ to be effective. Current implementation needs 150+ events with semantic backend for reliable Hurst estimation.

Table 3. Comparison of LLM personalization approaches at the local AI memory constraint boundary.

5. Pipeline Architecture

The pipeline operates in five sequential stages. Each stage has a defined input contract, output contract, and failure mode. The stages are:

Stage	Name	Input	Output	Key Mechanism
1	Encode	BehavioralSequence list	EncodedSequence list	Hash projection / SentenceTransformer embedding (128-dim vectors, L2-normalized)
2	Novelty Gate	Encoded events	Novel event set + weighted buffer	MinHash LSH (128 permutations, Jaccard threshold 0.92). $O(1)$ per event. Discards structurally redundant interactions.
3	Hierarchical Cluster	Novel event set	4-level cluster tree	HDBSCAN at 4 resolution levels: domain → category → pattern → micro. No pre-specified cluster count.
4	Self-Similarity Detect	Cluster tree	SelfSimilarPattern list + corpus H	Cross-level centroid cosine similarity series. R/S Hurst exponent. Patterns with $H \geq 0.70$, depth ≥ 3 = strong fractal rules.
5	Grammar Extract	Patterns + tree	GrammarRuleset (JSON) + JSONL	One GrammarRule per strong pattern. Rule cost = $O(1)$. Member cost = $O(N)$. Compression = $N / \text{rule_cost}$.

Table 4. Five-stage pipeline. Each stage is independently testable and replaceable.

5.1 Stage 2: Novelty Gate in Detail

The MinHash LSH novelty gate is the most computationally consequential stage. It processes each incoming event in $O(1)$ time regardless of corpus size, using a locality-sensitive hash structure. The gate computes a 128-permutation MinHash signature for each event and queries a banded LSH index for similar existing signatures. If Jaccard similarity to any existing event exceeds 0.92, the event is classified as redundant and discarded — its frequency weight is added to the matching existing entry rather than stored separately. Only novel events enter the clustering stage.

This means memory cost scales with behavioral novelty, not usage volume. A user who asks about invoice status 200 times in structurally similar ways adds one stored event with weight 200, not 200 stored events. This is the first compression stage and typically removes 40–60% of raw events in domain-constrained logs.

5.2 Stage 3: Hierarchical Clustering in Detail

HDBSCAN is chosen over K-Means and Gaussian Mixture Models for three reasons: (1) it does not require pre-specifying the number of clusters, which is unknown for a given user's behavioral space; (2) it is density-aware and handles irregular cluster shapes; (3) it natively identifies noise points — events that do not belong to any cluster — which become grammar residuals rather than being forced into poor cluster assignments.

Clustering runs at four HDBSCAN parameter levels corresponding to four behavioral resolution scales. Level 0 (coarsest) captures broad domains. Level 3 (finest) captures micro-behavioral signatures. Each level produces a set of clusters with centroid vectors and member lists. Parent-child relationships between levels are established via member fingerprint overlap when direct parent ID links are unavailable.

5.3 Stage 4: Hurst Exponent Computation in Detail

For each coarse-level cluster, the detector traces its descendants through finer resolution levels by following child clusters whose centroid cosine similarity to the parent exceeds a threshold (default 0.60 with semantic embeddings, 0.30 with hash projection). The cross-level similarity values form a series. Hurst exponent is computed over this series via rescaled-range (R/S) analysis:

$$R/S = (\max(\text{cumulative deviation}) - \min(\text{cumulative deviation})) / \text{std}(\text{series})$$

$$H = \log(R/S) / \log(n)$$

For series of length less than 4 (typical with 4-level clustering), the short-series proxy is used: $H = 0.5 + 0.5 \times \text{mean}(\text{similarities})$. This proxy underestimates H for strongly persistent processes and overestimates for weakly anti-persistent ones. The full R/S estimator becomes reliable at series length ≥ 4 , which requires a minimum corpus that sustains non-noise clusters at all four resolution levels — approximately 150 events with a semantic backend.

6. Empirical Results

6.1 Primary Benchmark

Pipeline run on a 53-event behavioral log from an accounting domain AI assistant (invoice queries, GL coding, bank reconciliation, noise). Backend: hash projection (64-dim). No GPU required.

Metric	Value
Input events	53
Events post novelty gate	18
Novelty gate compression	66%
Corpus Hurst exponent	0.601
Grammar rules extracted	2
Rule R0 Hurst / compression	0.907 / 6.25×
Rule R1 Hurst / compression	0.866 / 8.75×
Total compression (grammar vs. raw log)	30×
Residuals (uncompressed noise events)	16
Coverage (events explained by grammar)	100%
End-to-end latency (CPU)	70ms
Grammar output size	<4KB JSON

Table 5. Primary benchmark results. 53-event corpus, hash projection backend.

Important caveat: the corpus Hurst of 0.601 was computed on 53 events with the hash-projection backend. Stress testing (Section 7) established that the hash backend produces false-positive grammar rules at corpus sizes below ~150 events. The individual rule Hurst values (0.907, 0.866) are more reliable signals than the corpus-level value because they are computed over single-cluster similarity series rather than the full cross-level corpus series.

6.2 Minimum Viable Signal

With noise fixed at 10 events and structured events swept from 2 to 32, the minimum structured event count for grammar extraction is 10 events at 50% noise background. Below 10 structured events, H returns exactly 0.5 (the pipeline's fallback for series too short for R/S analysis).

Structured Events	Corpus H	Rules	Grammar Viable
2	0.500	0	No

Structured Events	Corpus H	Rules	Grammar Viable
4	0.500	0	No
6	0.500	0	No
8	0.500	0	No
10	0.651	2	Yes
12	0.647	2	Yes
16	0.559	2	Yes
20	0.553	2	Yes

Table 6. Minimum viable signal sweep. Noise fixed at 10 events. Grammar becomes viable at 10 structured events.

6.3 Entropy Gradient

Behavioral entropy swept from 100% structured to 0% structured in 11 steps. Key findings: pure-signal corpus H (0.659) exceeds pure-noise H (0.623), confirming directional sensitivity. Compression ratio peaks at 70–80% signal (not 100%) due to MinHash deduplication removing high-redundancy structured events before compression is measured — a measurement artifact, not a compression quality failure.

7. Stress Tests and Falsifying Findings

Five adversarial tests were designed before publication to establish a falsifiable baseline. Three of eleven test cases produced results that qualify the original claims. These are reported in full.

Finding 1 — False positives at small n. The pipeline extracted 2–3 grammar rules from purely random input at $n = 50$ –80 events using the hash-projection backend. The cause: geometric collisions in 64-dimensional hash space produce spurious clusters. At $n = 200$, the effect disappears ($H = 0.543$, 0 rules). Implication: hash backend results are unreliable below 150 events. The SentenceTransformer backend (semantically meaningful geometry) eliminates this failure mode.

Finding 2 — No grammar collapse under noise injection. Binary search for the noise injection rate causing grammar collapse found no collapse even at 99% noise. The pipeline produced 2–3 spurious rules at all noise levels. This confirms Finding 1: the hash backend does not have a well-defined failure boundary. The collapse test is meaningful only with the semantic backend.

Finding 3 — Anti-persistent sequences not separable from structured. The H gap between a 3-cluster structured corpus and a mean-reverting anti-persistent corpus was only 0.018 — statistically indistinguishable. The hash backend loses semantic anti-persistence in projection. This separation test requires a semantic embedding backend where geometric distance preserves semantic distance.

Implication: All quantitative claims in this paper apply to the hash-projection backend at the stated corpus size. The hypothesis itself — that behavioral logs exhibit fractal structure and grammar compression is viable — remains untested on real human-AI interaction logs. That is the next required experiment. The library provides the instrument. The community provides the logs.

8. The Open Experiment

The single highest-value external validation of this hypothesis is reproducible and low-effort for any developer with an existing LLM assistant in production:

1. Extract a log of ≥ 200 interactions from a domain-constrained LLM assistant.
2. Install the library: `pip install fractal-grammar`
3. Run with the SentenceTransformer backend (requires ~90MB model download).
4. Report: corpus H , n_{rules} , compression ratio, and domain.

If $H > 0.70$ and $n_{\text{rules}} \geq 2$: the hypothesis is supported on real data. Grammar compression is applicable to your interaction logs.

If $H < 0.55$ and $n_{\text{rules}} = 0$: the hypothesis is falsified for your domain. The interaction log does not have sufficient fractal structure for grammar compression. Publish this result — it is equally valuable.

Minimal reproduction:

```
from fractal_grammar import FractalGrammarPipeline, PipelineConfig, from_raw_texts
from fractal_grammar.embeddings.encoder import Backend
config = PipelineConfig(
    embedding_backend=Backend.SENTENCE_TRANSFORMER, model="all-MiniLM-L6-v2",
    similarity_threshold=0.6, min_hurst=0.55, )
pipe = FractalGrammarPipeline(config) # Load
your interaction log as a list of strings with open("my_interaction_log.txt") as f:
texts = [line.strip() for line in f if line.strip()]
sequences = [from_raw_texts([t]) for t in texts]
ruleset = pipe.run(sequences)
print(pipe.report())
ruleset.save_json("grammar.json")
```

9. Architecture Roadmap

The current release (v0.1.0) implements the grammar extraction layer. The full local AI personalization pipeline requires two additional components not yet implemented:

Component	Status	Notes
Grammar extraction (v0.1.0)	Implemented	This paper. MinHash + HDBSCAN + Hurst + GrammarRuleset.
LoRA training signal synthesis	Planned — v0.2.0	Generate weighted LoRA training pairs from grammar rules. GrammarRuleset.to_jsonl() stub exists.

Component	Status	Notes
Compaction scheduler (WAL pattern)	Planned — v0.3.0	Separate capture (continuous, $O(1)$) from compaction (scheduled, batch). Delta checkpointing.
SentenceTransformer validation suite	Planned — v0.2.0	Re-run all stress tests with semantic backend. Establish reliable Hurst thresholds on real data.
Selective forgetting (GDPR)	Research problem	Exact deletion from trained adapter is unsolved. Most underappreciated hard problem in this architecture.

Table 7. Architecture roadmap. v0.1.0 implements the grammar extraction layer only.

10. Viability Scorecard

Layer	Status	Notes
Local inference (frozen base model)	Green	Works now. Ollama, LocalAI, Jan, LM Studio all viable.
MinHash LSH novelty gating	Green	Works now. $O(1)$ per event. Validated in stress tests.
Grammar extraction (hash backend)	Yellow	Works at $n \geq 150$. False positives at $n < 100$.
Grammar extraction (semantic backend)	Yellow	Architecture correct. Stress tests not yet run on semantic backend.
LoRA delta stack (O-LoRA / Janus-LoRA)	Yellow	Underlying LoRA technology proven. Catastrophic forgetting risk with standard LoRA.
Full pipeline integration	Orange	1–2 years. Contention management and selective forgetting are the hard problems.
Fractal grammar extraction (production)	Orange	2–4 years for full production maturity. Approximation buildable now.
Selective forgetting (GDPR compliance)	Red	Unsolved research problem. Exact deletion from trained adapter not reliably achievable.

Table 8. Viability scorecard by pipeline layer.

11. Conclusion

This paper introduces fractal behavioral grammar extraction as a compression-first approach to local AI personalization. The central argument: behavioral interaction logs possess measurable fractal self-similarity (Hurst exponent), and this structure enables a compact generative grammar to represent behavioral history more efficiently than storing raw interaction logs.

The novel contribution is a specific pipeline composition — MinHash LSH novelty gating, four-level HDBSCAN hierarchical clustering, and R/S Hurst analysis across resolution levels — applied for the first time to LLM interaction logs as a compression criterion, motivated by the neuroscientific distinction between episodic memory (CA1) and rule-based memory (mPFC).

The implementation is released as fractal-grammar v0.1.0 under MIT license. The empirical stress tests, including three falsifying findings, are published alongside this paper as `stress_test_results.json`. The hypothesis remains to be validated on real human-AI interaction logs with the SentenceTransformer backend. That experiment is the next required step, and the community is invited to run it.

References

-
- Mandelbrot, B. B., & Wallis, J. R. (1969). Robustness of the rescaled range R/S in the measurement of noncyclic long run statistical dependence. *Water Resources Research*, 5(5).
- Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *PAKDD 2013*.
- Broder, A. Z. (1997). On the resemblance and containment of documents. *Proceedings of the Compression and Complexity of Sequences 1997*.
- Hu, E. J., et al. (2021). LoRA: Low-rank adaptation of large language models. *arXiv:2106.09685*.
- CA1 fractal spike timing and learning rate: PMC10217776. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10217776/>
- PocketLLM — On-device fine-tuning for personalized LLMs: [ACL Anthology 2024.privatenlp-1.10](#)
- QVAC Fabric (Tether) — cross-platform LoRA fine-tuning on consumer hardware, March 2026: huggingface.co/blog/qvac/fabric-llm-finetune
- App converts local LLM corrections into LoRA data (May 2026): aiweekly.co/alerts/app-converts-local-llm-corrections-into-lora-data
- Source code and stress test results: github.com/ryandmoore1976/fractal-grammar
-

This document constitutes a public prior art disclosure as of June 2026. All claims are made by the author in good faith. The author makes no warranty regarding patentability. A provisional patent application covering the novel pipeline composition described herein is recommended before filing third-party claims on this work.